1 표지 — 트랙 개시 · 관리



발표 대본 표지 · 약 25초

안녕하십니까. 이번 과제의 주제는 트랙 개시와 트랙 관리입니다.

지난 과제에서는 표적의 참값에 잡음을 더해 플롯을 만들고 클러터도 함께 만들었습니다. 이번에는 그 플롯을 데이터 처리 쪽으로 보내고, 받은 플롯 가운데 같은 표적에서 나온 것으로 보이는 두 개를 이어 트랙을 시작한 다음, 트랙 스코어로 트랙의 상태를 관리했습니다.

2 목차

INDEX

트랙 개시 · 관리
TrackMgr

Janghwan Kim (Harley)

01 트랙 개시 3
02 트랙 관리 9
03 구현 13
04 결과 22

2

발표 대본 목차 · 약 30초

순서는 네 부분입니다.

먼저 트랙 개시에서 플롯 한 개로 시작하는 방법과 두 개로 시작하는 방법을 비교하고, 이번에 쓴 2 Point 개시의 식과 속도 게이트를 설명드리겠습니다. 이어서 트랙 관리에서 M/N 방식과 트랙 스코어 방식을 보겠습니다.

그다음 이 내용을 어떻게 구현했는지 코드로 보고, 마지막으로 프로그램 결과 화면과 개시 결과를 보겠습니다.

01. 트랙 개시

트랙이 무엇인지 보고, 플롯 두 개로 트랙을 시작하는 2 Point 개시와 속도 게이트를 설명합니다.

1.1 트랙과 트랙 개시 4

1.2 1-Point 와 2-Point 개시 5

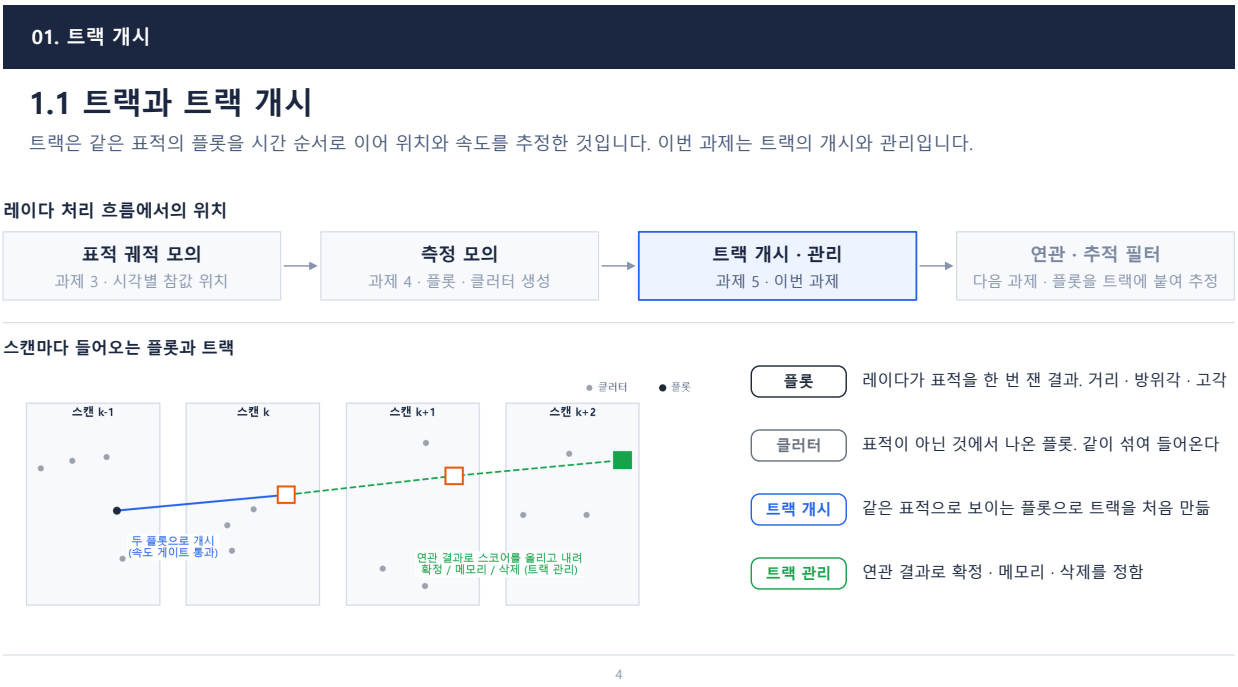
1.3 2-Point 초기 상태변수와 오차 공분산 6

1.4 속도 게이팅 7

1.5 측정 공분산을 VC-NED 로 옮기기 8

발표 대본 트랙 개시 · 약 15초

먼저 트랙 개시입니다. 트랙이 무엇이고 레이다 처리 흐름에서 어디에 있는지 본 다음, 1-Point 와 2-Point 개시를 비교하고, 2-Point 개시의 상태변수와 오차 공분산, 속도 게이트, 측정 공분산 순서로 설명드리겠습니다.



발표 대본

트랙 개시 · 약 65초

위쪽 흐름에서 이번 과제의 위치를 먼저 보겠습니다. 과제 3 에서 표적 궤적을 만들었고, 과제 4 에서 그 궤적을 재서 플롯과 클러터를 만들었습니다. 이번 과제는 그 플롯을 받아 트랙을 시작하고 관리하는 부분이고, 트랙에 다음 플롯을 붙이는 연관과 추적 필터는 다음 과제에서 다룹니다.

아래 그림처럼 스캔마다 플롯이 들어오는데, 그중에는 표적 플롯도 있고 클러터도 섞여 있습니다. 트랙은 같은 표적에서 나온 플롯을 시간 순서로 이어서 표적의 위치와 속도를 추정한 것입니다. 트랙을 처음 만드는 것을 트랙 개시라고 하고, 만든 트랙이 진짜 표적인지 판단해서 확정하거나 지우는 것을 트랙 관리라고 합니다.

이번 과제는 앞 스캔과 이번 스캔의 플롯 두 개로 트랙을 개시하고, 트랙 스코어로 상태를 관리합니다.

5 1.2 1-Point 와 2-Point 개시

01. 트랙 개시

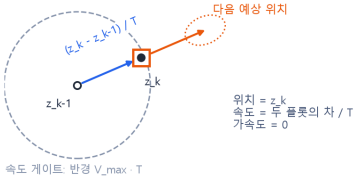
1.2 1-Point 와 2-Point 개시

플롯 하나로 바로 시작하면 속도를 모르고, 연속 두 스캔의 플롯으로 시작하면 위치 변화로 속도를 구할 수 있습니다.

1-Point: 플롯 하나로 개시



2-Point: 연속 두 스캔의 플롯으로 개시



1. **1-Point 개시**
   장점: 플롯 하나로 바로 개시 (빠름)
   단점: 속도를 몰라 초기 속도 오차가 크다
2. **2-Point 개시**
   장점: 두 플롯의 위치 변화로 속도를 구해 1-Point 보다 속도 오차가 작다
   단점: 연속 두 스캔이 필요해 개시가 늦다
3. **속도 게이트**
   두 플롯이 같은 표적인지 먼저 거른다 (1.4)

이번 과제 2 Point 트랙 개시, 속도 게이트 V_max = 800 m/s, 트랙 ID 100 ~ 10000, 초기 상태변수 · 오차 공분산의 가속도 항은 0 (과제 값)

5

발표 대본

트랙 개시 · 약 70초

트랙을 시작하는 방법은 두 가지입니다.

왼쪽의 1-Point 개시는 플롯 하나만으로 트랙을 만듭니다. 위치는 플롯 위치를 그대로 쓰는데, 플롯 하나로는 표적이 어느 쪽으로 가는지 알 수 없어서 속도를 0 으로 두고 속도 오차를 크게 잡습니다. 플롯이 들어오자마자 트랙을 만들 수 있다는 것이 장점이고, 초기 속도 오차가 크다는 것이 단점입니다.

오른쪽의 2-Point 개시는 연속된 두 스캔의 플롯을 씁니다. 위치는 이번 스캔 플롯이고, 속도는 두 플롯 위치의 차이를 스캔 간격으로 나눠 구합니다. 그래서 속도 오차를 줄일 수 있지만, 두 스캔을 기다려야 해서 개시가 한 스캔 늦습니다. 두 플롯이 같은 표적에서 나왔는지는 점선 원으로 그린 속도 게이트로 걸러 냅니다.

이번 과제는 2-Point 개시를 적용했습니다.

1.3 2-Point 초기 상태변수와 오차 공분산

01. 트랙 개시

1.3 2-Point 초기 상태변수와 오차 공분산

위치는 이번 플롯, 속도는 두 플롯의 차 / T, 가속도는 0 입니다. 오차 공분산은 두 플롯의 측정 공분산 R 에서 바로 나옵니다.

상태변수 9 개 (VC-NED: x 북, y 동, z 아래)

$$X_k = [\, x \;\; x' \;\; x'' \;\; y \;\; y' \;\; y'' \;\; z \;\; z' \;\; z'' \,]^T$$

축마다 초기값

1	**위치**	z_k	이번 스캔 플롯
2	**속도**	(z_k − z_k-1) / T	두 플롯의 차 / 스캔 간격
3	**가속도**	0	두 점으로는 알 수 없다 (과제 조건)

오차 공분산 (축 a, b 의 3 x 3 칸마다)

$$P_{pp} = R_k$$
$$P_{pv} = P_{vp} = R_k / T$$
$$P_{vv} = (R_k + R_{k-1}) / T^2$$

위치 오차 = w_k
속도 오차 = (w_k − w_k-1) / T
가속도 행 · 열 = 0

w 는 측정 오차, R = 그 공분산 (1.5). 두 오차를 빼서 T 로 나누기 때문에 속도 분산은 두 R 의 합 / T² 이 된다.

	x	x'	x''	y	y'	y''	z	z'	z''
x	943	9432	0	1148	1.1e4	0	5	53	0
x'	9432	1.9e5	0	1.1e4	2.3e5	0	53	-1006	0
x''	0	0	0	0	0	0	0	0	0
y	1148	1.1e4	0	1454	1.5e4	0	-4	-43	0
y'	1.1e4	2.3e5	0	1.5e4	2.9e5	0	-43	796	0
y''	0	0	0	0	0	0	0	0	0
z	5	53	0	-4	-43	0	2535	2.5e4	0
z'	53	-1006	0	-43	796	0	2.5e4	5.1e5	0
z''	0	0	0	0	0	0	0	0	0

표적 5 예 (5.8 km, T = 0.1 s). 단위 m², m²/s, m²/s²

발표 대본

트랙 개시 · 약 80초

2-Point 개시로 만드는 트랙의 상태변수는 북, 동, 아래 세 축마다 위치, 속도, 가속도를 둔 9 개 값입니다.

축마다 보면 위치는 이번 플롯, 속도는 두 플롯의 차를 스캔 간격 T 로 나눈 값이고, 가속도는 두 점으로는 구할 수 없어서 과제 조건대로 0 으로 둡니다.

오차 공분산은 플롯의 측정 오차에서 바로 나옵니다. 위치 오차는 이번 플롯의 오차 그대로라서 분산이 R_k 입니다. 속도 오차는 두 플롯 오차의 차를 T 로 나눈 것이라서 분산이 R_k 더하기 R_k-1 을 T 제곱으로 나눈 값이 되고, 위치와 속도 사이 공분산은 R_k 나누기 T 입니다.

오른쪽 그림은 표적 5 를 5.8 km 에서 개시했을 때 실제로 계산된 9 x 9 행렬입니다. 파란 칸이 위치, 주황 칸이 속도, 초록 칸이 위치와 속도 사이 값이고, 가속도 행과 열은 모두 0 입니다. 측정 오차가 축끼리 상관이 있어서 축 사이 칸도 채워집니다. 속도 분산이 19 만 정도라 속도 표준편차가 400 m/s 를 넘는데, 이 점은 결과에서 다시 보겠습니다.

01. 트랙 개시

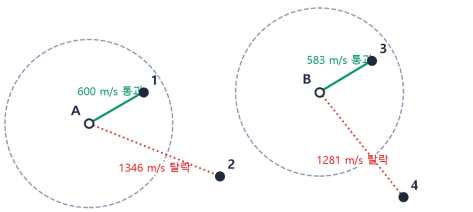
1.4 속도 게이팅

두 플롯이 같은 표적이면 한 스캔 동안 움직인 거리가 V_max · T 를 넘을 수 없습니다. 이 조건을 넘는 쌍으로만 트랙을 개시합니다.

$$|\, z_{(k,i)} - z_{(k-1,j)} \,| \;/\; \Delta T \;<\; V_{max}$$

V_max = 800 m/s, ΔT = 0.1 s -> 앞 플롯에서 반경 80 m 안
i: 이번 스캔 플롯, j: 앞 스캔 플롯. 거리는 VC-NED 직교좌표로 잰다

짝짓기: 앞 스캔 플롯 (빈 원) 과 이번 스캔 플롯 (채운 원)



점선 원 = 반경 V_max · T = 80 m. 속력이 작은 쌍부터 하나씩 고르고, 쓴 플롯은 다시 쓰지 않는다



스캔 간격이 짧아 게이트가 작습니다. 가로 방향 잡음은 거리에 비례해서, 멀리 있는 표적은 참 표적 쌍도 자주 게이트를 넘지 못합니다 (4.4).

7

발표 대본

트랙 개시 · 약 80초

두 플롯이 같은 표적에서 나왔는지는 속도 게이트로 판단합니다. 두 플롯의 거리 차를 스캔 간격으로 나눈 속력이 표적이 낼 수 있는 최대 속력 V_max 보다 작아야 한다는 조건입니다. 과제 값은 800 m/s 이고, 스캔 간격이 0.1 초라서 앞 플롯 주변 반경 80 m 안에 이번 플롯이 있어야 합니다.

왼쪽 그림에서 A 와 1 은 600 m/s 라서 통과하고, A 와 2 는 1,346 m/s 라서 탈락합니다. 한 플롯이 여러 쌍에 걸릴 수 있어서, 통과한 쌍 가운데 속력이 가장 작은 쌍부터 하나씩 고르고, 이미 쓴 플롯은 다시 쓰지 않습니다.

오른쪽은 거리별로 두 플롯 차이의 잡음 크기를 게이트와 비교한 것입니다. 각도 오차는 거리를 곱해 가로 방향 위치 오차가 되기 때문에, 1 km 에서는 잡음이 게이트 안에 들어가지만 14 km 에서는 게이트보다 훨씬 커집니다. 그래서 참 표적 쌍도 14 km 에서는 열 번 중 한 번 정도만 통과합니다.

8 1.5 측정 공분산을 VC-NED 로 옮기기

01. 트랙 개시

1.5 측정 공분산을 VC-NED 로 옮기기

플롯의 오차는 거리 · 방위각 · 고각에 있어서, 트랙 좌표계 (VC-NED) 의 위치 공분산 R 로 바꿔야 오차 공분산을 채울 수 있습니다.

1 **측정 표준편차 (과제 4, SNR 20)**

$\sigma_r = 4.74$ m, $\sigma_{az} = 0.478°$, $\sigma_{el} = 0.494°$

2 **안테나 직교좌표로**

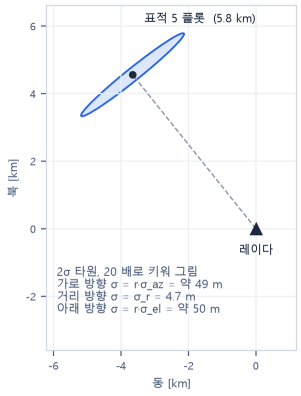
$R_{ant} = J \cdot \mathrm{diag}(\sigma_r^2, \sigma_{az}^2, \sigma_{el}^2) \cdot J^T$

3 **VC-NED 로**

$R = M \cdot R_{ant} \cdot M^T$

J: 구좌표 (r, az, el) -> 안테나 직교 (x, y, z) 변환의 미분
M: 안테나 -> 동체 -> VC-NED 회전 (플랫폼 Roll · Pitch · Yaw)
회전만 들어가서 오차 타원의 크기는 그대로, 방향만 바뀐다

VC-NED 플랫폼 무게중심이 원점, 축은 북 · 동 · 아래. 자세가 바뀌어도 축이 돌지 않아 트랙을 이 좌표계에서 만든다



8

발표 대본

트랙 개시 · 약 65초

앞에서 본 R 은 위치 측정 오차의 공분산인데, 플롯은 안테나 기준 거리, 방위각, 고각으로 들어오기 때문에 이것을 트랙 좌표계로 옮겨야 합니다.

트랙은 VC-NED 에서 만듭니다. VC-NED 는 플랫폼 무게중심이 원점이고 축은 북, 동, 아래로 고정된 좌표계라서, 플랫폼 자세가 바뀌어도 축이 돌지 않습니다.

먼저 거리, 방위각, 고각의 표준편차를 야코비안 J 로 안테나 직교좌표의 공분산으로 바꾸고, 안테나에서 동체, 동체에서 VC-NED 로 가는 회전 M 을 앞뒤로 곱합니다.

오른쪽 그림은 표적 5 의 플롯에서 계산한 공분산을 20 배로 키워 그린 것입니다. 거리 방향 오차는 4.7 m 인데, 가로 방향은 거리 5.8 km 에 방위각 오차를 곱해 약 49 m 가 되어서, 레이다를 바라보는 선에 수직으로 길쭉한 타원이 됩니다.

02. 트랙 관리

만든 트랙을 확정할지 지울지 정하는 M/N 방식과 트랙 스코어 방식을 설명합니다.

2.1 M/N 방식 10
2.2 트랙 스코어 방식 11
2.3 Assigned_Flag 에 따른 스코어 변화 12

발표 대본 트랙 관리 · 약 15초

다음은 트랙 관리입니다. 만든 트랙을 확정할지 지울지 판단하는 두 가지 방법, M/N 방식과 트랙 스코어 방식을 보고, 이번 과제에서 쓴 트랙 스코어가 연관 결과에 따라 어떻게 바뀌는지 보겠습니다.



발표 대본

트랙 관리 · 약 50초

M/N 방식은 최근 N 번 가운데 몇 번 연관에 성공했는지를 셉니다.

임시 트랙은 위 그림처럼 N 번 중 M 번 연관에 성공하면 확정하고, M 번 연속으로 실패하면 지웁니다. 예에서는 다섯 번 중 네 번 성공했기 때문에 확정입니다.

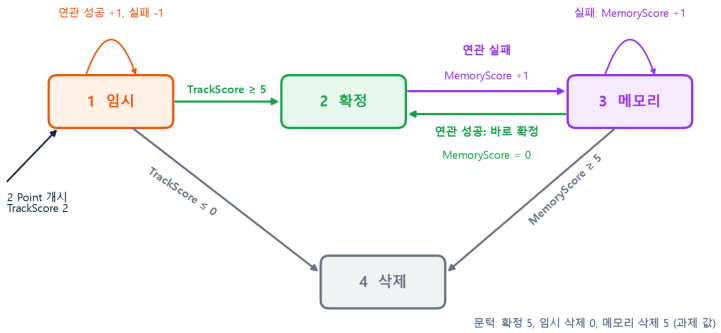
확정 트랙은 연관에 실패하면 메모리 트랙으로 바꾸고, 다음 플롯이 붙지 않는 동안 예측 위치로 이어 갑니다. 메모리 상태가 정해 둔 시간보다 길게 이어지면 표적을 놓친 것으로 보고 지웁니다.

세기가 단순하다는 것이 장점이지만, 성공과 실패를 몇 번 했는지만 보기 때문에 트랙 상태를 값 하나로 나타내기는 어렵습니다.

02. 트랙 관리

2.2 트랙 스코어 방식

트랙마다 TrackScore 와 MemoryScore 를 두고, 점수가 문턱을 넘으면 상태 (TrackState) 를 바꿉니다.



1. **TrackState (과제 값)**
   1 임시, 2 확정, 3 메모리, 4 삭제
2. **문턱 (과제 값)**
   임시 트랙 확정 5, 임시 트랙 삭제 0
   메모리 트랙 삭제 5
3. **메모리 트랙이 다시 연관되면**
   바로 확정 트랙으로, MemoryScore = 0
4. **과제에 없어 정한 값**
   개시 직후 TrackScore 2 (두 플롯으로 만든 트랙)
   한 번에 올리고 내리는 양 1

11

발표 대본

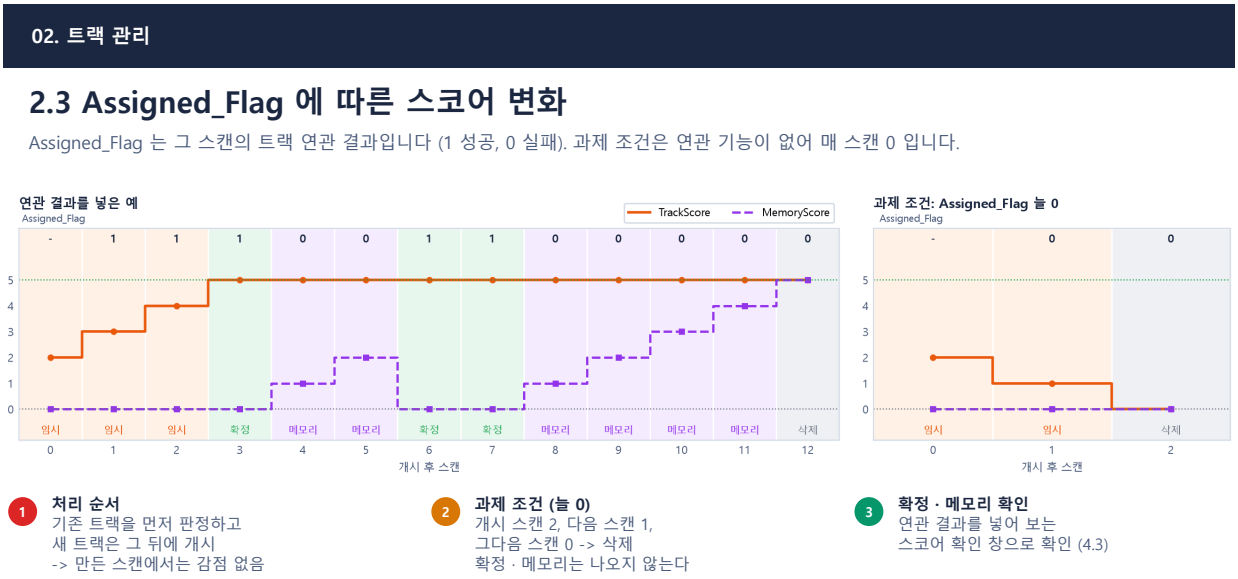
트랙 관리 · 약 80초

이번 과제에서 쓴 트랙 스코어 방식입니다. 트랙마다 TrackScore 와 MemoryScore 두 점수를 두고, 상태는 1 임시, 2 확정, 3 메모리, 4 삭제의 네 가지입니다.

임시 트랙은 연관에 성공하면 TrackScore 를 1 올리고 실패하면 1 내립니다. 5 이상이 되면 확정하고 0 이하가 되면 삭제합니다.

확정 트랙이 연관에 실패하면 메모리 트랙으로 바꾸고 MemoryScore 를 1 올립니다. 메모리 트랙은 연관에 성공하면 바로 확정으로 돌아가면서 MemoryScore 를 0 으로 되돌리고, 계속 실패해서 MemoryScore 가 5 이상이 되면 삭제합니다.

세 문턱 값 5, 0, 5 는 과제 값입니다. 과제에 정해져 있지 않은 두 가지는 제가 정했습니다. 개시 직후 Track Score 는 두 플롯으로 만든 트랙이라 2 로 두었고, 한 번에 올리고 내리는 양은 1 로 두었습니다.



발표 대본

트랙 관리 · 약 75초

점수가 실제로 어떻게 바뀌는지 두 경우로 보겠습니다. 위쪽 숫자가 스캔마다 넣은 Assigned_Flag 이고, 1 이 연관 성공, 0 이 실패입니다.

왼쪽은 연관 결과를 직접 넣어 본 예입니다. 2 에서 시작한 TrackScore 가 세 번 성공해 5 가 되면서 확정되고, 두 번 실패하면 메모리로 바뀌며 MemoryScore 가 1, 2 로 오릅니다. 다시 성공하면 바로 확정으로 돌아가고 MemoryScore 는 0 이 됩니다. 마지막에는 다섯 번 연속 실패해서 MemoryScore 가 5 가 되는 스캔에 삭제됩니다.

오른쪽이 이번 과제 조건입니다. 아직 연관 기능이 없어서 매 스캔 Assigned_Flag 를 0 으로 두기 때문에, 트랙은 개시한 스캔에 2, 다음 스캔에 1, 그다음 스캔에 0 이 되어 삭제됩니다. 기존 트랙을 먼저 판정하고 새 트랙은 그 뒤에 만들기 때문에, 새 트랙은 만든 스캔에서는 점수가 깎이지 않습니다.

03. 구현

요구사항이 어느 코드에 있는지 정리하고, 쓰레드 · UDP 구조와 트랙 개시 · 관리 코드를 봅니다.

3.1 요구사항과 구현 14

3.2 쓰레드와 UDP 구조 15

3.3 플롯 메시지 STRUCT_PlotMsg 16

3.4 송신: 플롯 메시지 만들기 (f_Lnk_BuildMsg) 17

3.5 수신: 좌표 변환과 측정 공분산 (f_Lnk_ProcessMsg) 18

3.6 2 Point 개시: 쌍 고르기 (f_Trk_ProcessScan) 19

3.7 2 Point 초기화 (f_Trk_TwoPointInit) 20

3.8 트랙 스코어 로직 (f_Trk_UpdateState) 21

발표 대본 구현 · 약 15초

이제 구현입니다. 과제 요구사항이 어느 코드에 있는지 먼저 정리하고, 쓰레드와 UDP 구조, 플롯 메시지, 송신과 수신, 그리고 트랙 개시와 스코어 로직 코드를 순서대로 보겠습니다.

03. 구현

3.1 요구사항과 구현

과제 3 장 (구현) 의 항목을 구현한 곳입니다. 트랙 개시 · 관리는 TrackMgr.c, 플롯 메시지 송수신은 PlotLink.c 에 있습니다.

과제 요구사항	구현	파일	쪽
2 Point 트랙 개시, 상태변수 X_k 9 개	`f_Trk_TwoPointInit`	TrackMgr.c	20
mat_EstX, mat_EstP 가속도 항 0	Matrix_initialize 후 위치 · 속도 칸만 채움	TrackMgr.c	20
속도 게이팅 V_max = 800 m/s	`f_Trk_ProcessScan, TRK_VMAX`	TrackMgr.c	19
트랙 ID 100 ~ 10000	`f_Trk_NewId`	TrackMgr.c	19
트랙 스코어: 확정 5, 삭제 0, 메모리 삭제 5	`f_Trk_UpdateState`	TrackMgr.c	21
TrackScore, MemoryScore, TrackState (1 ~ 4)	`ST_Track, TRK_STATE_*`	TrackMgr.h	21
안테나 구좌표 플롯 메시지를 수신 쓰레드로	`STRUCT_PlotMsg, f_Lnk_TxThread / RxThread`	Idd.h, PlotLink.c	15 ~ 18
표적 플롯 + 클러터, 플롯 ID 10000 ~ 1000000	`f_Lnk_BuildMsg`	PlotLink.c	17
UDP 또는 TCP/IP 전송	UDP (과제 1 소켓 함수의 UDP 부분)	PlotLink.c	15
매 스캔 Assigned_Flag = 0 (성공 1)	f_Trk_ProcessScan 첫 단계	TrackMgr.c	19
True, Plot, Track 심볼 2D · 3D 전시	`CTrajectoryPlot, CTrackView3D`	TrajectoryPlot.cpp	23 ~ 24

발표 대본

구현 · 약 55초

과제 요구사항과 구현을 한 표로 정리했습니다.

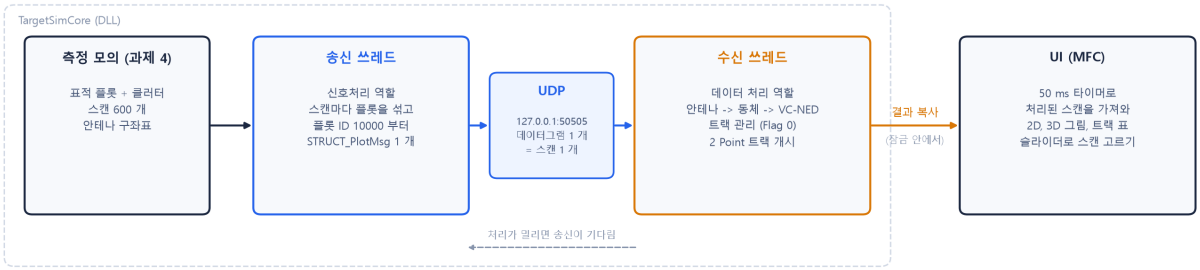
트랙 개시는 TrackMgr.c 에 있습니다. 2 Point 개시와 상태변수, 가속도 항 0 은 f_Trk_TwoPointInit, 속도 게이트와 트랙 ID 는 f_Trk_ProcessScan 과 f_Trk_NewId 에서 처리합니다. 트랙 관리의 세 문턱과 메모리 트랙 처리는 f_Trk_UpdateState 에 있습니다.

시뮬레이션 쪽은 PlotLink.c 에 있습니다. 송신 쓰레드가 안테나 구좌표 플롯 메시지를 만들어 UDP 로 보내고, 수신 쓰레드가 받아서 트랙 개시와 관리를 돌립니다. 결과 화면의 2D 와 3D 전시는 UI 의 두 그림 클래스가 맡습니다. 오른쪽 끝 숫자가 그 내용을 설명하는 쪽입니다.

03. 구현

3.2 쓰레드와 UDP 구조

과제 4 의 측정 모의 결과를 신호처리가 내보내는 플롯으로 보고, 송신 쓰레드가 스캔마다 UDP 로 보내 수신 쓰레드 (데이터 처리) 가 처리합니다.



1 **송신 쓰레드**
과제 1 에서 Winsock2 로 옮긴 UDP 함수
스캔 하나 = 데이터그램 하나
수신 버퍼 8 MB, 밀리면 기다림

2 **수신 쓰레드**
수신 대기는 100 ms 마다 끊어
멈춤 신호를 확인
스캔 번호로 빠진 스캔을 셈

3 **UI**
50 ms 타이머로 처리된 스캔을
가져와 그림과 표를 그림
결과는 잠금 안에서 복사

15

발표 대본

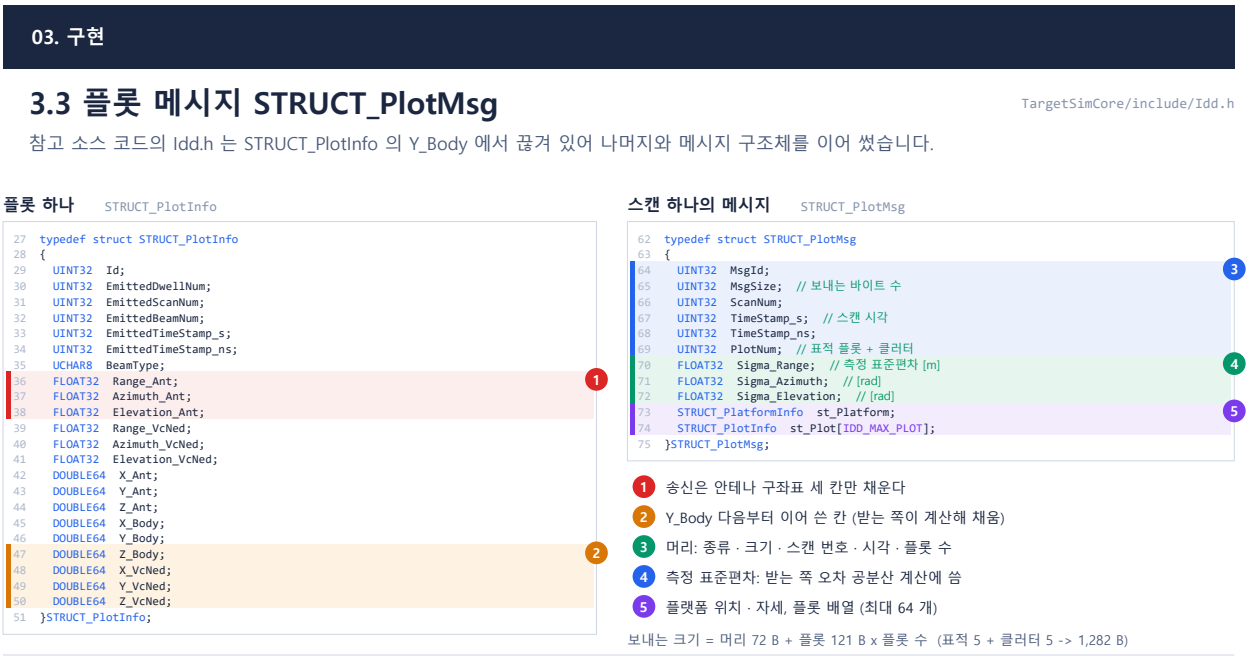
구현 · 약 75초

전체 구조입니다. 과제 4 의 측정 모의 결과를 레이다 신호처리가 내보내는 플롯으로 보고, 데이터 처리가 그것을 받는 형태로 만들었습니다.

송신 쓰레드는 스캔마다 표적 플롯과 클러터를 섞어 플롯 메시지 하나를 만들고 UDP 데이터그램 하나로 보냅니다. 소켓 함수는 과제 1 에서 Winsock2 로 옮겨 둔 UDP 함수를 그대로 가져왔습니다. UDP 는 보낸 크기 그대로 도착해서 메시지 하나를 한 번에 받을 수 있습니다.

수신 쓰레드가 데이터 처리 역할입니다. 메시지를 받아 플롯 좌표를 VC-NED 로 바꾸고, 트랙 관리와 2 Point 개시를 돌린 뒤 스캔마다 결과를 쌓아 둡니다. UDP 는 받는 쪽이 밀리면 소리 없이 버리기 때문에, 처리가 밀리면 송신이 기다리게 했고 메시지의 스캔 번호로 빠진 스캔을 셉니다.

UI 는 50 ms 마다 처리된 스캔을 가져와 그림과 표를 그립니다.



발표 대본

구현 · 약 75초

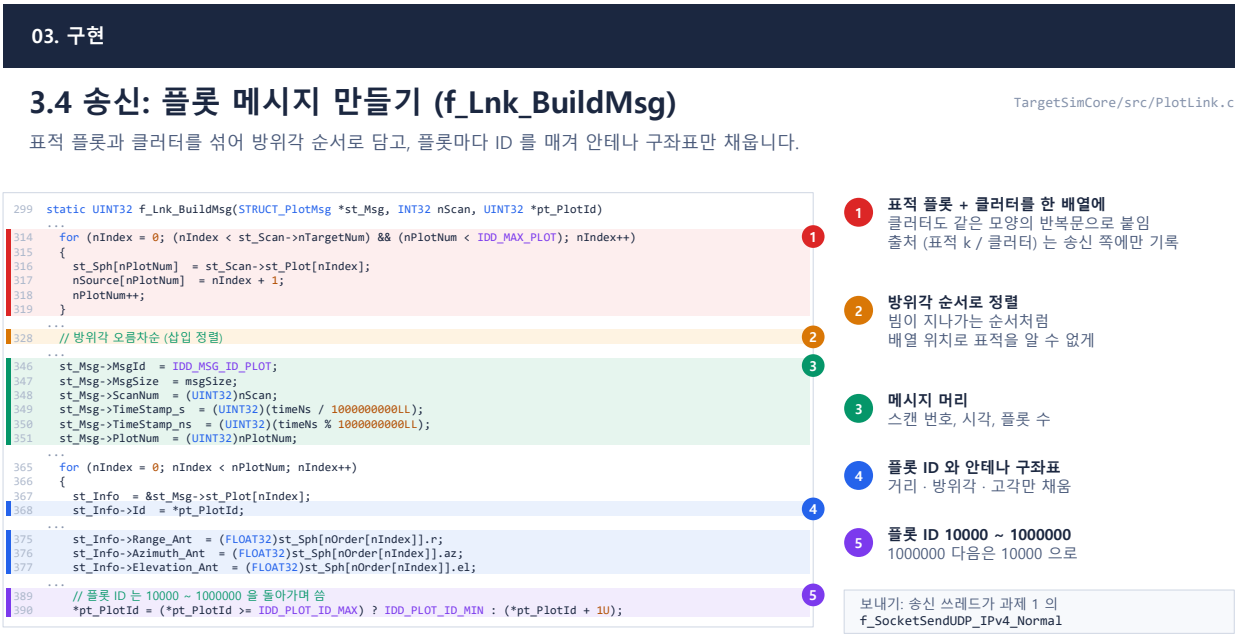
플롯 메시지입니다. 참고 소스 코드의 Idd.h 는 STRUCT_PlotInfo 의 Y_Body 까지만 있어서 나머지를 이어서 완성했습니다.

왼쪽에서 Y_Body 까지는 원래 그대로이고, 아래 네 줄이 이어 쓴 부분입니다. 구좌표가 안테나와 VC-NED 두 벌이라서 직교좌표도 안테나, 동체, VC-NED 세 벌이 되도록 Z_Body 와 VC-NED 직교 성분을 붙였습니다.

오른쪽 STRUCT_PlotMsg 는 머리 정보와 플롯 배열입니다. 머리에는 메시지 종류와 크기, 스캔 번호와 시각, 플롯 수, 측정 표준편차, 플랫폼 정보가 들어갑니다. 측정 표준편차는 받는 쪽이 오차 공분산을 계산하는 데 쓰고, 플랫폼 자세는 좌표 변환에 씁니다.

송신 쪽은 과제 조건대로 안테나 구좌표 성분만 채우고, 나머지 좌표 칸은 받는 쪽이 계산해서 채웁니다. 보낼 때는 플롯 수만큼만 보내서, 표적 5 개와 클러터 5 개면 메시지 하나가 1,282 바이트입니다.

3.4 송신: 플롯 메시지 만들기



발표 대본

구현 · 약 70초

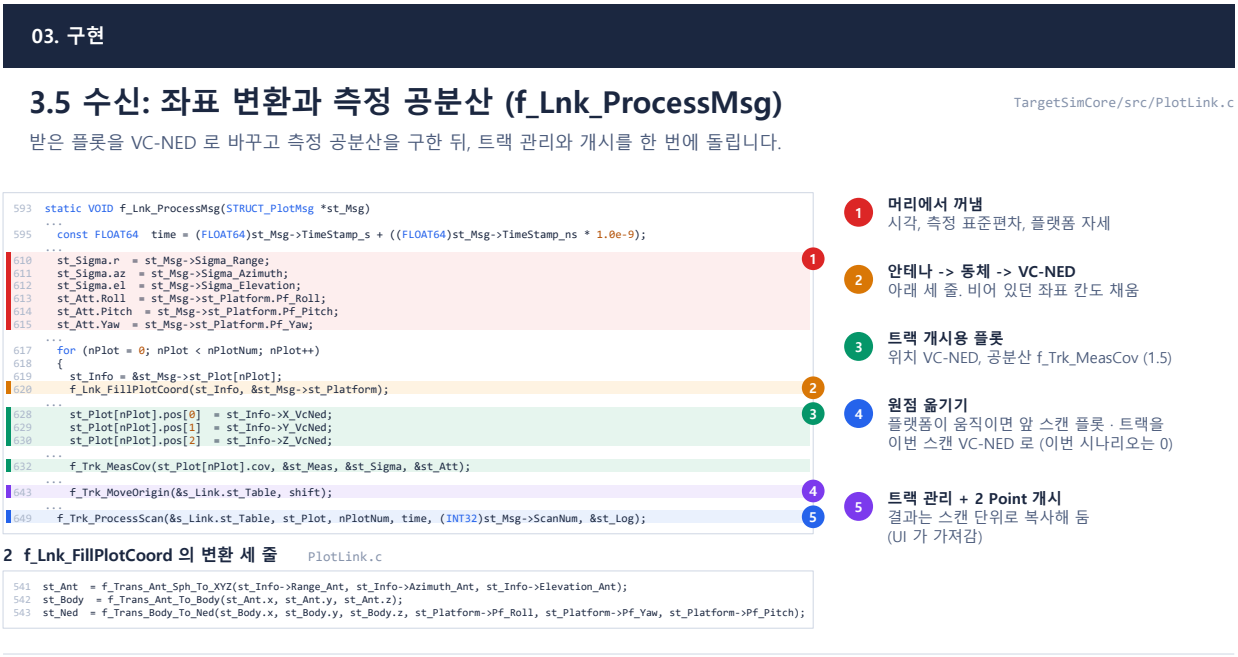
송신 쪽에서 메시지를 만드는 f_Lnk_BuildMsg 입니다.

먼저 그 스캔의 표적 플롯과 클러터를 한 배열에 모으고, 방위각 순서로 정렬합니다. 빔이 지나가는 순서대로 플롯이 나오는 것처럼 보이게 하려는 것이고, 이렇게 하면 배열 위치만 보고 표적과 클러터를 구분할 수 없습니다.

다음으로 메시지 머리에 스캔 번호와 시각, 플롯 수, 측정 표준편차와 플랫폼 정보를 채웁니다.

플롯마다 ID 를 매기고 안테나 기준 거리, 방위각, 고각을 넣습니다. ID 는 과제 조건대로 10000 부터 하나씩 올리고 1000000 다음에는 10000 으로 돌아갑니다. 어느 ID 가 표적이었는지는 메시지에 넣지 않고 송신 쪽에만 따로 적어 두었다가, 결과 화면에서 확인할 때만 씁니다.

마지막으로 송신 쓰레드가 과제 1 의 UDP 송신 함수로 메시지를 보냅니다.



발표 대본

구현 · 약 85초

받는 쪽, 데이터 처리의 중심인 f_Lnk_ProcessMsg 입니다.

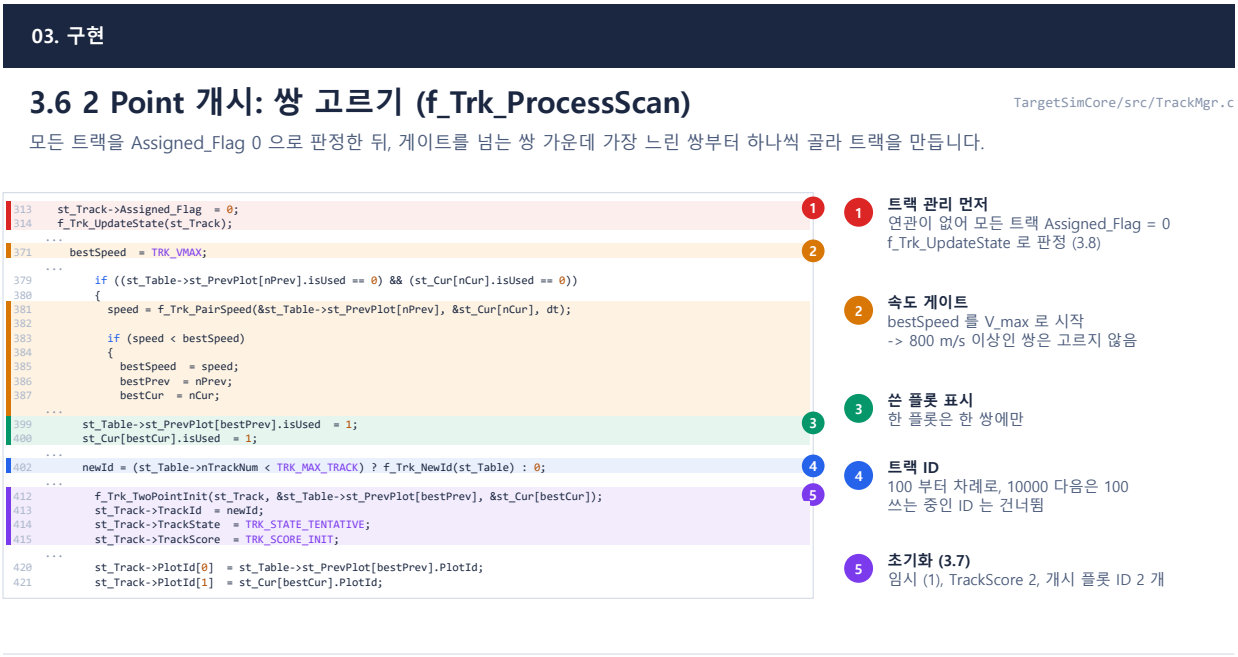
먼저 메시지 머리에서 시각, 측정 표준편차, 플랫폼 자세를 꺼냅니다.

플롯마다 f_Lnk_FillPlotCoord 로 좌표를 바꿉니다. 아래 세 줄처럼 안테나 구좌표를 안테나 직교좌표로, 동체로, 플랫폼 자세를 써서 VC-NED 로 옮기고, 비어 있던 메시지 칸을 채웁니다. 변환 함수는 지난 과제들과 같은 좌표변환 코드입니다.

그다음 트랙 개시에 넣을 플롯을 만듭니다. 위치는 VC-NED 이고, 측정 공분산은 앞에서 본 식대로 f_Trk_MeasCov 로 계산합니다.

VC-NED 는 원점이 플랫폼을 따라 움직이기 때문에, 플랫폼이 움직이면 앞 스캔 플롯과 이번 스캔 플롯의 원점이 서로 다릅니다. 그래서 처리하기 전에 f_Trk_MoveOrigin 으로 앞 스캔 플롯과 트랙 위치를 이번 스캔 기준으로 옮겨 둡니다. 이렇게 해야 두 플롯의 차가 플랫폼에 대한 상대 속도가 아니라 표적 속도가 됩니다. 이번 시나리오는 플랫폼이 서 있어서 옮기는 양은 0 입니다.

마지막으로 f_Trk_ProcessScan 을 불러 트랙 관리와 개시를 한 번에 돌리고, 그 결과를 스캔 단위로 결과 버퍼에 복사해 둡니다.



발표 대본

구현 · 약 80초

스캔 하나를 처리하는 f_Trk_ProcessScan 입니다.

첫 번째 부분이 트랙 관리입니다. 아직 연관 기능이 없어서 모든 트랙의 Assigned_Flag 를 0 으로 두고 스코어 로직을 돌립니다.

두 번째 부분이 2 Point 개시입니다. 앞 스캔에서 아직 쓰지 않은 플롯과 이번 스캔 플롯의 모든 쌍에 대해 속력을 구하고, 그중 가장 작은 쌍을 찾습니다. bestSpeed 를 V_max 로 시작하기 때문에 속도 게이트를 넘지 못한 쌍은 처음부터 고르지 않습니다.

고른 쌍은 두 플롯 모두 사용 표시를 해서 다른 쌍에 다시 쓰이지 않게 하고, 새 ID 를 받아 트랙을 만듭니다. 상태는 임시, TrackScore 는 2 이고, 개시에 쓴 두 플롯의 ID 도 남깁니다. 이 과정을 고를 쌍이 없을 때까지 반복합니다.

트랙 ID 는 f_Trk_NewId 가 100 부터 차례로 주고, 10000 다음에는 100 으로 돌아가며 쓰는 중인 ID 는 건너뜁니다.

03. 구현

3.7 2 Point 초기화 (f_Trk_TwoPointInit)

TargetSimCore/src/TrackMgr.c

9 x 1 상태변수와 9 x 9 오차 공분산을 0 으로 만든 뒤, 축마다 위치 · 속도 칸만 채웁니다. 가속도 칸은 0 으로 남습니다.

```
164 VOID f_Trk_TwoPointInit(ST_Track *st_Track, const ST_TrkPlot *st_Prev, const ST_TrkPlot *st_Cur)
165 {
166   const FLOAT64  dt = st_Cur->time - st_Prev->time;
167   matrix  *st_X = &st_Track->mat_EstX;
168   matrix  *st_P = &st_Track->mat_EstP;
169   INT32  nAxis;
170   INT32  nAxisB;
171
172   Matrix_initialize(st_X, TRK_STATE_DIM, 1);                                          (1)
173   Matrix_initialize(st_P, TRK_STATE_DIM, TRK_STATE_DIM);
174
175   for (nAxis = 0; nAxis < 3; nAxis++)
176   {
177     st_X->e[(3 * nAxis)][0]  = st_Cur->pos[nAxis];                                    (2)
178     st_X->e[(3 * nAxis) + 1][0]  = (st_Cur->pos[nAxis] - st_Prev->pos[nAxis]) / dt;
179     st_X->e[(3 * nAxis) + 2][0]  = 0.0;
180
181     for (nAxisB = 0; nAxisB < 3; nAxisB++)
182     {
183       st_P->e[(3 * nAxis)][(3 * nAxisB)]  = st_Cur->cov[nAxis][nAxisB];               (3)
184       st_P->e[(3 * nAxis)][(3 * nAxisB) + 1]  = st_Cur->cov[nAxis][nAxisB] / dt;
185       st_P->e[(3 * nAxis) + 1][(3 * nAxisB)]  = st_Cur->cov[nAxis][nAxisB] / dt;
186       st_P->e[(3 * nAxis) + 1][(3 * nAxisB) + 1]  = (st_Cur->cov[nAxis][nAxisB] + st_Prev->cov[nAxis][nAxisB]) / (dt * dt);
187     }
188   }
189 }
```

(1) **0 으로 채운 행렬**
참고 소스 코드의 Matrix_initialize
가속도 칸은 손대지 않아 0

(2) **상태변수 (축마다)**
위치 = 이번 플롯
속도 = 위치 차 / 시간 차, 가속도 = 0

(3) **오차 공분산 (축 a, b)**
P_pp = R_k, P_pv = P_vp = R_k / T
P_vv = (R_k + R_k-1) / T²
축 사이 칸까지 같은 식 (1.3)

인덱스
축 a 의 위치 3a, 속도 3a+1, 가속도 3a+2
x 북 0 ~ 2, y 동 3 ~ 5, z 아래 6 ~ 8

20

발표 대본

구현 · 약 60초

트랙의 초기 상태변수와 오차 공분산을 만드는 f_Trk_TwoPointInit 입니다.

먼저 참고 소스 코드의 행렬 함수로 9 x 1 상태변수와 9 x 9 오차 공분산을 만들고 모두 0 으로 채웁니다. 그래서 가속도 칸은 따로 손대지 않아도 0 으로 남습니다.

축마다 위치에는 이번 플롯 위치를, 속도에는 두 플롯 위치 차를 시간 차로 나눈 값을 넣습니다.

안쪽 반복문은 오차 공분산을 채웁니다. 축 a 와 축 b 의 3 x 3 칸마다 위치는 R_k, 위치와 속도 사이는 R_k 나누기 T, 속도는 R_k 더하기 R_k-1 을 T 제곱으로 나눈 값을 넣습니다. 1.3 에서 본 식과 같고, 축 사이 칸까지 같은 식으로 채웁니다.

03. 구현

3.8 트랙 스코어 로직 (f_Trk_UpdateState)

TargetSimCore/src/TrackMgr.c

상태마다 case 하나씩입니다. 임시는 TrackScore 로, 확정 · 메모리는 MemoryScore 로 다음 상태를 정합니다.

임시 트랙

```
108 VOID f_Trk_UpdateState(ST_Track *st_Track)
109 {
110   const INT32 isAssigned = (st_Track->Assigned_Flag != 0) ? 1 : 0;
111
112   switch (st_Track->TrackState)
113   {
114   case TRK_STATE_TENTATIVE:
115     st_Track->TrackScore = st_Track->TrackScore + ((isAssigned != 0) ? 1 : -1);   (1)
116
117     if (st_Track->TrackScore >= TRK_SCORE_CONFIRM)                                (2)
118     {
119       st_Track->TrackState = TRK_STATE_CONFIRMED;
120     }
121     else if (st_Track->TrackScore <= TRK_SCORE_DELETE)
122     {
123       st_Track->TrackState = TRK_STATE_DELETED;
124     }
125     else
126     {
127       // 임시 트랙 유지
128     }
129     break;
130
```

확정 트랙, 메모리 트랙

```
131   case TRK_STATE_CONFIRMED:
132     if (isAssigned == 0)
133     {
134       st_Track->TrackState  = TRK_STATE_MEMORY;                                   (3)
135       st_Track->MemoryScore  = st_Track->MemoryScore + 1;
136     }
137     break;
138
139   case TRK_STATE_MEMORY:
140     if (isAssigned != 0)
141     {
142       st_Track->TrackState  = TRK_STATE_CONFIRMED;                                (4)
143       st_Track->MemoryScore  = 0;
144     }
145     else
146     {
147       st_Track->MemoryScore  = st_Track->MemoryScore + 1;
148
149       if (st_Track->MemoryScore >= TRK_MEMORY_DELETE)                             (5)
150       {
151         st_Track->TrackState = TRK_STATE_DELETED;
152       }
153     }
154     break;
155
```

(1) 성공 +1, 실패 -1 (2) 5 이상 확정, 0 이하 삭제 (3) 확정 -> 메모리, MemoryScore +1 (4) 메모리 -> 바로 확정, MemoryScore 0 (5) MemoryScore 5 이상 삭제

21

발표 대본

구현 · 약 60초

트랙 스코어 로직 f_Trk_UpdateState 입니다. 상태마다 switch 의 case 가 하나씩 있습니다.

임시 트랙은 Assigned_Flag 가 1 이면 TrackScore 를 1 올리고 0 이면 1 내린 다음, 5 이상이면 확정, 0 이하면 삭제로 바꿉니다.

확정 트랙은 연관에 실패하면 메모리로 바꾸고 MemoryScore 를 1 올립니다.

메모리 트랙은 연관에 성공하면 바로 확정으로 돌리면서 MemoryScore 를 0 으로 되돌리고, 실패하면 MemoryScore 를 1 올린 뒤 5 이상이면 삭제로 바꿉니다.

삭제 상태가 된 트랙은 다음 스캔을 시작할 때 트랙 표에서 빠지고, 그 ID 는 다시 쓸 수 있게 됩니다.

04. 결과

프로그램 화면과 송수신 결과, 2D · 3D 전시, 트랙 수명, 거리에 따른 개시 결과를 봅니다.

4.1 실행 화면과 송수신 결과 23

4.2 True · Plot · Track 2D · 3D 전시 24

4.3 트랙 수명과 스코어 확인 25

4.4 거리에 따른 개시 결과 26

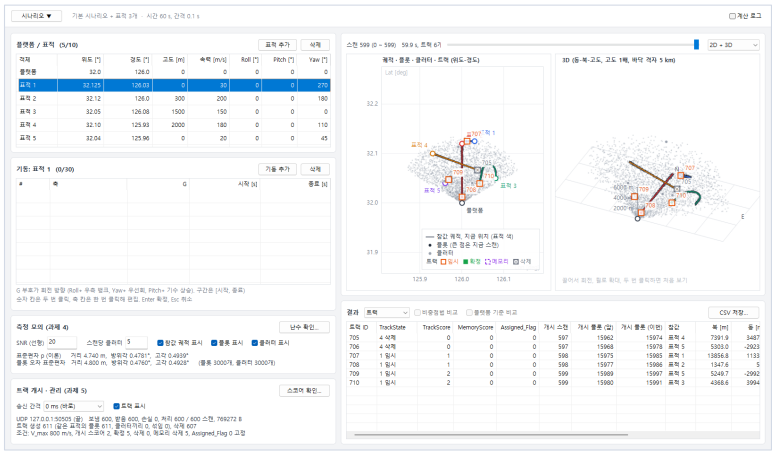
발표 대본 결과 · 약 15초

다음은 결과입니다. 프로그램 화면과 송수신 결과, 2D 와 3D 전시, 트랙의 수명과 스코어 확인을 보고, 마지막으로 거리에 따라 개시 결과가 어떻게 달라지는지 보겠습니다.

04. 결과

4.1 실행 화면과 송수신 결과

기본 시나리오 + 표적 3 개, 60 s, 스캔 600 개, 스캔당 클러터 5 개 (과제 4 조건). 실행하면 바로 송수신을 시작합니다.



1. **UDP 송수신**
   600 스캔 보냄, 600 스캔 받음, 손실 0
   메시지 1,282 B x 600 = 769,200 B

2. **트랙 611 개 개시, 607 개 삭제**
   개시 쌍은 모두 같은 표적의 플롯
   클러터끼리 0, 섞임 0 (송신 쪽 기록과 대조)

3. **스캔 슬라이더**
   고른 스캔의 2D, 3D 그림과 트랙 표
   송신 간격 100 ms 면 실시간으로 따라감

4. **트랙 표**
   트랙 ID, TrackState, TrackScore, MemoryScore, Assigned_Flag, 위치 · 속도

23

발표 대본

결과 · 약 70초

프로그램을 실행하면 바로 나오는 화면입니다. 과제 4 와 같이 기본 시나리오에 표적 세 개를 더한 표적 다섯 개, 60 초, 스캔당 클러터 다섯 개 조건입니다.

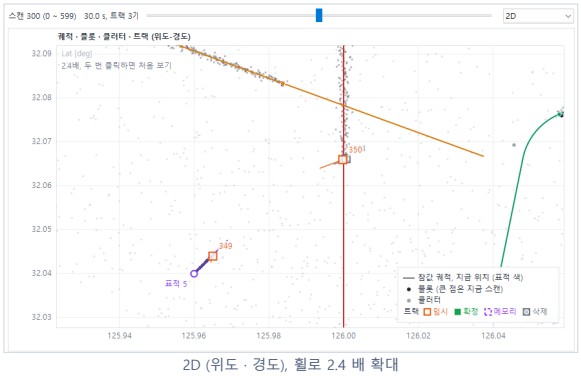
왼쪽 아래 트랙 개시 · 관리 칸을 보시면, UDP 로 600 스캔을 보내 600 스캔을 받았고 빠진 스캔은 없습니다. 트랙은 611 개가 만들어졌는데, 개시에 쓴 두 플롯이 모두 같은 표적에서 나온 것이었고, 클러터끼리나 표적과 클러터가 섞인 쌍은 없었습니다. 클러터는 넓은 범위에 흩어져 있어서 80 m 안에 두 개가 연달아 들어올 일이 드물기 때문입니다.

위쪽 슬라이더로 스캔을 고르면 2D, 3D 그림과 아래 트랙 표가 그 스캔으로 바뀝니다. 송신 간격을 100 ms 로 두면 실제 스캔 간격과 같은 속도로 보내서, 트랙이 생기고 지워지는 모습을 따라 볼 수 있습니다.

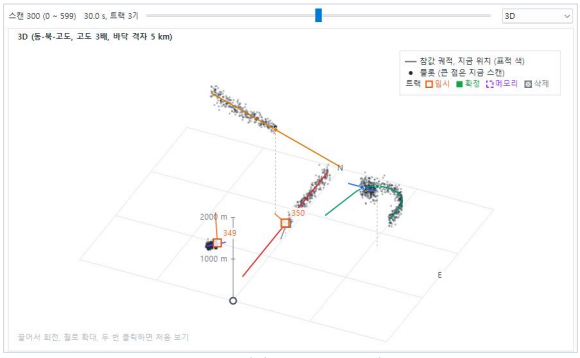
04. 결과

4.2 True · Plot · Track 2D · 3D 전시

같은 스캔 (300, 30 s) 을 2D 와 3D 로 그린 화면입니다. 참값, 플롯, 트랙 심볼을 함께 그립니다.



2D (위도 · 경도), 휠로 2.4 배 확대



3D, 클러터를 끄고 고도 3 배

참값 선: 궤적, 마름모: 지금 위치 (표적 색)

트랙 빈 주황 사각형: 임시, 회색 X: 삭제, 숫자: ID

플롯 옅은 점: 지난 스캔, 흰 테 큰 점: 지금 스캔

속도 선 트랙 속도로 1 초 동안 가는 거리

발표 대본

결과 · 약 65초

과제에서 요구한 True, Plot, Track 심볼을 함께 그린 화면입니다. 왼쪽이 2D, 오른쪽이 3D 입니다.

선은 참값 궤적이고, 마름모는 지금 스캔의 참값 위치입니다. 옅은 점은 지난 스캔의 플롯과 클러터이고, 흰 테를 두른 큰 점이 지금 스캔에 받은 플롯입니다. 사각형이 트랙인데, 빈 주황 사각형이 임시 트랙, 회색 X 사각형이 삭제 상태가 된 트랙이고, 옆 숫자가 트랙 ID 입니다. 사각형에서 뻗은 선은 트랙 속도로 1 초 동안 가는 거리입니다.

2D 는 휠로 확대해서 표적 주변을 볼 수 있고, 3D 는 끌어서 돌려 볼 수 있습니다. 3D 에서는 고도를 읽을 수 있도록 참값 위치에서 바닥까지 점선을 내렸습니다.

속도 선의 방향이 참값 궤적과 많이 어긋나는 것이 보이는데, 그 이유는 4.4 에서 보겠습니다.

04. 결과

4.3 트랙 수명과 스코어 확인

과제 조건 (Assigned_Flag 늘 0) 에서 트랙은 세 스캔 동안만 보입니다. 확정 · 메모리 상태는 스코어 확인 창으로 확인했습니다.

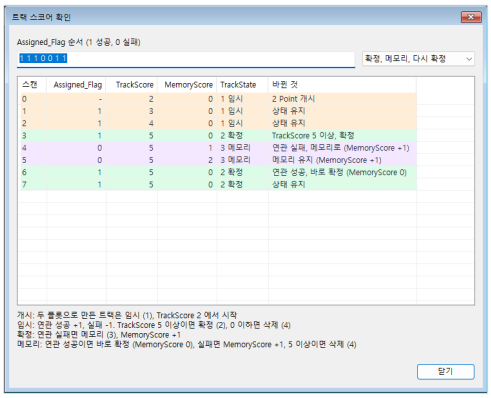
스캔 300 의 트랙 표

결과 트랙 비중점법 비교 플랫폼 기준 비교 CSV 저장...

트랙 ID	TrackState	TrackScore	MemoryScore	Assigned_Flag	개시 스캔	개시 플롯 (앞)	개시 플롯 (이번)	참값	폭 [m]	중 [r
348	4 삭제	0	0	0	298	12974	12985	표적 2	7344.5	62
349	1 임시	1	0	0	299	12989	12999	표적 5	4881.5	-3301
350	1 임시	2	0	0	300	12997	13005	표적 2	7315.0	-23

1 **나이가 다른 트랙 세 개**
350: 이번 스캔에 개시, TrackScore 2
349: 한 스캔 전 개시, 1 / 348: 두 스캔 전 개시, 0 -> 삭제 (4)

2 **모든 트랙이 같은 수명**
개시 2 -> 1 -> 0 (삭제), 다음 스캔에 표에서 빠짐
611 개 모두 이 순서 (확인: 스캔마다 TrackScore = 2 - 나이)



스코어 확인 창: 1 1 1 0 0 1 1 을 넣은 결과

발표 대본

결과 · 약 65초

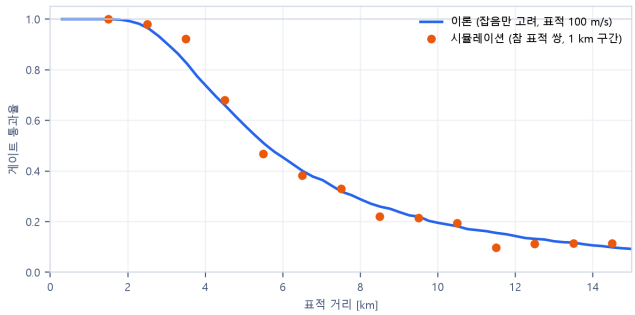
왼쪽은 300 번 스캔의 트랙 표입니다. 같은 스캔에 나이가 다른 트랙 세 개가 보입니다. 350 번은 이번 스캔에 만들어져 TrackScore 가 2 이고, 349 번은 한 스캔 전에 만들어져 1, 348 번은 두 스캔 전에 만들어져 0 이 되면서 삭제 상태가 되었습니다. Assigned_Flag 는 모두 0 입니다. 과제 조건에서는 모든 트랙이 이렇게 세 스캔 동안만 보이고 사라집니다.

그래서 확정과 메모리 상태는 시뮬레이션에서 나오지 않고, 오른쪽의 스코어 확인 창에서 확인했습니다. 연관 결과를 직접 넣으면 같은 스코어 함수를 그대로 불러서 스캔마다 점수와 상태를 보여 줍니다. 1 1 1 0 0 1 1 을 넣으면 세 번째 스캔에 확정되고, 두 번 실패하면 메모리, 다시 성공하면 MemoryScore 가 0 으로 돌아가며 확정되는 것을 볼 수 있습니다.

04. 결과

4.4 거리에 따른 개시 결과

각도 오차는 거리에 비례해 위치 오차가 됩니다. 스캔 간격 0.1 s 에서 게이트 반경은 80 m 라 먼 표적일수록 개시가 드뭅니다.



표적	거리 [km]	게이트 통과	개시 트랙	속도 오차 RMS
1	13.9 ~ 14.1	11 %	58	554 m/s
2	13.3 -> 1.4	43 %	158	475 m/s
3	6.1 ~ 10.9	25 %	115	575 m/s
4	8.4 ~ 13.1	20 %	98	531 m/s
5	5.8 ~ 6.0	45 %	182	549 m/s

게이트 통과: 참 표적 쌍 기준. 개시 트랙: 한 번 쓴 플롯은 다시 쓰지 않아 최대 300. 속도 오차: 개시 순간 추정 속도와 참 속도 차

속도 오차가 큰 이유
위치 오차를 0.1 s 로 나누기 때문. 스캔 간격이 수 초인 탐색 레이다라면 같은 위치 오차에서 속도 오차가 수십 분의 일로 준다.

1 **SNR 1000 이면**
모든 표적이 두 스캔마다 개시 (각 300 개)
정규화 속도 오차 제곱 평균 2.8 ~ 2.9 (이론 3) -> P 가 실제 오차와 맞다

발표 대본

결과 · 약 85초

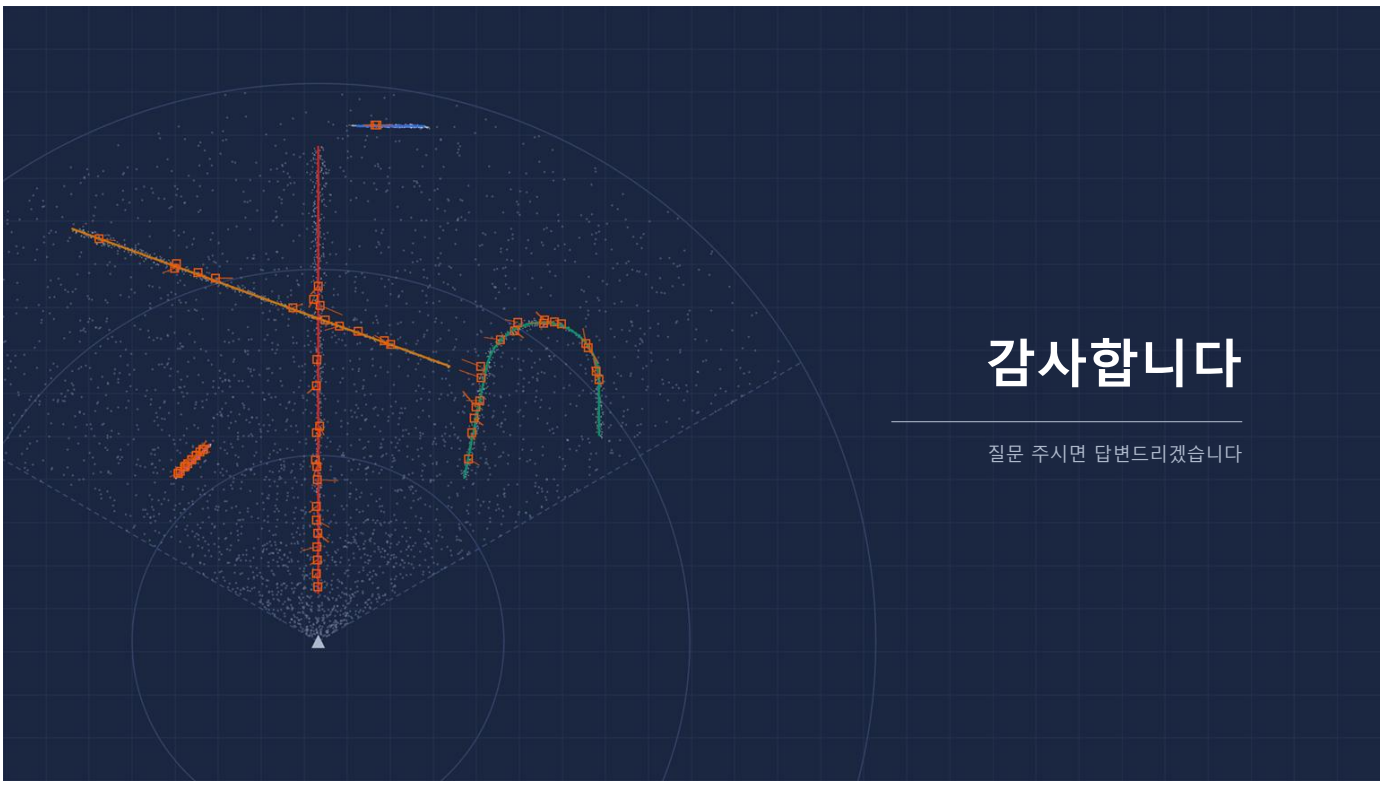
마지막으로 표적마다 트랙이 얼마나 개시됐는지 보겠습니다.

왼쪽 그래프는 참 표적 쌍이 속도 게이트를 통과한 비율을 거리별로 그린 것입니다. 1 km 안쪽은 거의 다 통과하지만 5 km 에서 절반, 14 km 에서는 열 번 중 한 번 정도로 떨어집니다. 파란 선이 잡음만으로 계산한 이론 값이고, 주황 점이 시뮬레이션 결과로 거의 같습니다. 각도 오차가 거리에 비례해 위치 오차가 되는데, 스캔 간격이 0.1 초라 게이트 반경이 80 m 로 작기 때문입니다.

오른쪽 표처럼 14 km 에 있는 표적 1 은 트랙이 58 번, 6 km 근처의 표적 5 는 182 번 개시됐습니다. 개시된 트랙의 속도 오차도 500 m/s 안팎으로 큽니다. 위치 오차를 0.1 초로 나누기 때문이고, 스캔 간격이 수 초인 탐색 레이다라면 같은 위치 오차에서도 속도 오차가 수십 분의 일로 줄어듭니다.

SNR 을 1000 으로 올리면 모든 표적이 두 스캔마다 개시되고, 속도 오차를 오차 공분산으로 나눠 제곱한 값의 평균이 이론값 3 에 가까운 2.8 정도가 나옵니다. 그래서 계산한 오차 공분산이 실제 오차 크기와 맞는다는 것도 확인했습니다.

27 감사합니다



발표 대본 마무리 · 약 10초

이상으로 발표를 마치겠습니다. 감사합니다. 질문 주시면 답변드리겠습니다.